



Práctica Colab: Autopsia a un Tokenizador

Demuestra que dominas el "idioma nativo" de los Transformers

1. Preparativos Obligatorios (Setup)

Antes de empezar a codificar los retos, asegúrate de tener tu entorno listo en Google Colab. Revisa que cumples todos estos puntos:

- ☐ Abre un nuevo cuaderno en Google Colab.
- ☐ Conéctate a una GPU (Entorno de ejecución > Cambiar tipo de entorno > T4 GPU).
- ☐ Instala las librerías necesarias con: `!pip install -q --upgrade datasets transformers huggingface_hub`
- ☐ Haz login con tu Token de Hugging Face. Importa `login` desde `huggingface_hub` y usa la variable de entorno `HF_TOKEN`.
- ☐ **Importante:** Asegúrate de que tu cuenta tiene acceso aprobado a la familia Llama 3.1 en Hugging Face.

2. Misiones de Código (Los Retos)

Reto 1: El Traductor Universal (Encode)

Instancia el tokenizador del modelo `meta-llama/Meta-Llama-3.1-8B`. Una vez lo tengas guardado en una variable, coge la siguiente frase exacta y tradúcela a la lista de números que entendería la red neuronal:

"Los LLMs no entienden palabras. Solo entienden matemáticas."

Imprime el resultado por pantalla.

 **Pista de código:** Utiliza la clase `AutoTokenizer.from_pretrained()`. Recuerda añadir `trust_remote_code=True` como parámetro para evitar advertencias de seguridad. Luego, aplica el método `.encode()` a la variable de texto.

Reto 2: Ingeniería Inversa (Decode y Sub-palabras)

Toma la lista de números enteros (IDs) que acabas de generar en el Reto 1 y realiza las siguientes dos acciones en celdas separadas:

1. **Descodifica la secuencia completa** para recuperar la frase original. Observa la salida impresa: *¿Qué token especial ha inyectado el modelo automáticamente al principio del texto?*
2. **Descodifica la secuencia "lote a lote"** (token a token). Busca la palabra "matemáticas" en la lista devuelta. *¿En cuántos fragmentos se ha dividido la palabra?*

💡 **Pista de código:** Para recuperar el texto completo y continuo, usa el método `.decode(tokens)`. Para ver las piezas sueltas que componen la frase, usa `.batch_decode(tokens)`.

Reto 3: El Director y el Actor (Plantilla de Chat)

Los modelos de instrucción necesitan una estructura muy particular. Carga el tokenizador de la versión `meta-llama/Meta-Llama-3.1-8B-Instruct`.

Diseña un diccionario de Python estándar con dos mensajes:

- **Rol System:** "Eres un profesor de Inteligencia Artificial muy sarcástico."
- **Rol User:** "¿Me explicas qué es exactamente un tensor?"

Aplica la plantilla del modelo a estos mensajes y muestra el texto final estructurado que se enviaría a la red neuronal.

 **Pista de código:** Crea tu lista de diccionarios así: `mensajes = [{"role": "system", "content": "..."}, {...}]`. Después, utiliza el método `.apply_chat_template(mensajes, tokenize=False, add_generation_prompt=True)` y haz un `print()` del resultado.

3. Choque de Titanes

Reto Final: Modelos de Vanguardia y el Secreto del Código

El tokenizador de Llama 3.1 es genial para inglés, pero... ¿por qué los modelos enfocados en programación son mucho mejores estructurando código? Todo radica en la eficiencia de su diccionario.

Descarga los tokenizadores de los modelos Open Source más demandados del momento en Hugging Face:

- **Qwen 2.5 Coder (Alibaba):** `Qwen/Qwen2.5-Coder-7B-Instruct`
- **Phi-4 (Microsoft):** `microsoft/Phi-4-mini-instruct`
- **DeepSeek V3 (DeepSeek AI):** `deepseek-ai/DeepSeek-V3`

Declara una variable de texto con un pequeño bloque de código en Python (con tabulaciones y saltos de línea). Usa el método `encode()` en cada uno de estos tokenizadores (y en el de Llama 3.1) para comprobar la longitud de tokens que genera cada uno.

Finalmente, haz un bucle `for` imprimiendo cada token individual con `decode()` para el modelo **Qwen Coder**. Observa cómo tokeniza los espacios en blanco y la sintaxis de programación de forma radicalmente diferente a Llama.

 **Pista de código:** Define tu código así:

```
codigo = "def hello_world(person):\n print('Hello', person)"
```

Para imprimir la autopsia del token de Qwen usa:

```
tokens = qwen_tokenizer.encode(codigo)
```

```
for t in tokens: print(f"{t} = {qwen_tokenizer.decode(t)}")
```

4. Entregable

¿Qué debes entregar?

Una vez termines todos los retos en tu cuaderno de Google Colab, asegúrate de haber ejecutado todas las celdas para que se vean los resultados debajo de cada bloque de código.

Ve al menú superior: **Archivo > Descargar > Descargar .ipynb**.
Sube ese archivo descargado a la plataforma de entregas del curso.